# COMP 2011 Midterm Exam - Fall 2025 - HKUST

Date: October 26, 2025 (Sunday)
Time Allowed: 2 hours, 2:30-4:30pm
Instructions:

1. This is a closed-book, closed-notes examination.
2. There are **7** questions on **15** pages (including this cover page, honor code).
3. Write your answers in the space provided in black/blue ink. NO pencil please, otherwise you are not allowed to appeal for any grading disagreements.
4. All programming codes in your answers must be written in the ANSI C++ 2011 version as taught in the class.
5. For programming questions, you are **NOT** allowed to define additional helper functions or structures, nor global variables unless otherwise stated. You **cannot** use any library functions not mentioned, nor the `auto` keyword in defining identifiers in the questions.

| | |
|---|---|
| Student Name | |
| Student ID | |
| Email Address | |
| Lecture & Lab Section | |

| **Problem** | **Score** |
|---|---|
| 1: Switch | / 6 |
| 2: Loops | / 6 |
| 3: Scope | / 6 |
| 4: Struct | / 8 |
| 5: Function | / 14 |
| 6: 2D array | / 20 |
| 7: Recursion | / 20 |
| Total | / 80 |

**The HKUST Academic Honor Code**

Honesty and integrity are central to the academic work of HKUST. Students of the University must observe and uphold the highest standards of academic integrity and honesty in all the work they do throughout their program of study.

As members of the University community, students have the responsibility to help maintain the academic reputation of HKUST in its academic endeavors.

Sanctions will be imposed if students are found to have violated the regulations governing academic integrity and honesty.

Declaration of Academic Integrity

I confirm that I have answered the questions using only materials specifically approved for use in this examination, that all the answers are my own work, and that I have not received any assistance during the examination.

Signature: _____

**Problem 1 [6 points] C++ Basics: Switch**

Consider the following program:

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        switch (i) {
            case 2: cout << 2;
            case 3: cout << 3; continue;
            case 4: cout << 4; return 0;
            case 5: cout << 5; break;
            default: break;
        }
    }
    return 0;
}
```

Can this program be compiled successfully? If so, please write the output after executing the program. If not, please identify the line numbers where the compilation errors occur and explain.

**Problem 2 [6 points] C++ Basics: Loops**

Consider the following program:

```
1  #include <iostream>
2  using namespace std;
3
4  void printSomething(int n) {
5      for (int line = 1; line <= n; line++) {
6          int C = 1;
7          for (int i = 1; i <= line; i++) {
8              cout << C;
9              C = C * (line - i) / i;
10         }
11         cout << endl;
12     }
13 }
14
15 int main()
16 {
17     printSomething(4);
18     return 0;
19
20 }
```

Can this program be compiled successfully? If so, please write the output after executing the program. If not, please identify the line numbers where the compilation errors occur and explain.

**Problem 3 [6 points] Scope**

Consider the following program:

```
1  #include <iostream>
2  using namespace std;
3
4  int x = 1;
5
6  int foo(int x) {
7    cout << x++ << endl;
8    return x;
9  }
10
11 void bar(int& x) {
12   cout << ++x << endl;
13 }
14
15 int main() {
16   x = foo(x);
17
18   for (int i = 0, x = 2; i < 3; i++) {
19     x = foo(x);
20     bar(x);
21     if (x > 3)
22       break;
23   }
24
25   bar(x);
26
27   return 0;
28 }
```

Can this program be compiled successfully? If so, please write the output after executing the program. If not, please identify the line numbers where the compilation errors occur and explain.

**Problem 4 [8 points] Struct**

(a) (4 points) What is the output of the following code:

```
#include <iostream>
using namespace std;

struct Student {
    string name;
    int score;
};

void awardBonus(Student s) {
    s.score += 10;
}

void penalize(Student& s) {
    s.score -= 5;
}

int main() {
    Student alice = {"Alice", 80};
    Student bob = alice;
    awardBonus(bob);
    penalize(alice);
    cout << "Alice: " << alice.score << endl;
    cout << "Bob: " << bob.score << endl;

    return 0;
}
```

(b) (4 points) In the below code, there are some compilation errors. Write down the line number of the code that causes the error, and give a brief explanation of why the program cannot be compiled.

```
1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4
5  enum Dept { CSE, ECE, MATH };
6
7  struct Date {
8      unsigned int year;
9      unsigned int month;
10     unsigned int day;
11 };
12
13 struct Student_Record {
14     char name[32];
15     unsigned int id;
16     Dept dept;
17     Date entry;
18 };
19
20 int main() {
21     Student_Record s = { "Alice", 12345, CSE, {2025, 10, 15} };
22     s.name = "Bob";
23     cout << s.name << " " << s.id << " " << endl;
24     cout << s.year << "/" << s.month << "/" << s.day << endl;
25     return 0;
26 }
```

**Problem 5 [14 points] Function: Armstrong Number**

An Armstrong number (also known as a narcissistic number) for a given number of digits is a number that is equal to the sum of its own digits each raised to the power of the number of digits.

For example, the number 153 is an Armstrong number because it has three digits, and if you take each digit (1, 5, and 3), raise them to the power of three, and sum them, you get:

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$$

Conversely, the number 123 is not an Armstrong number because:

$$1^3 + 2^3 + 3^3 = 1 + 8 + 27 = 36,$$

which is not equal to 123.

Implement a C++ function that determines whether a given non-negative integer is an Armstrong number. Feel free to use $pow(a, b) = a^b$ provided by `cmath` library in your code.

*Sample Input/Output*

| **Input** | **Output** |
|---|---|
| 0 | Yes |
| 1 | Yes |
| 123 | No |
| 370 | Yes |
| 8208 | Yes |

**Problem 6 [20 points] 2D Array**

For this problem, you will implement two functions that handle 2D character arrays.

(a) (12 points) Write the function `readAndSplit`. Its purpose is to read a phrase from the keyboard, "split" the words it consists of, and store it into a 2D character array, where each row is a word. The phrase consists of English letters (capital and small case) and/or numbers. You can assume that:

- The total length of the input is at most 255 characters.
- Each word has at most 8 characters.
- Words are separated by a single empty space.
- There is no leading or trailing whitespace in the phrase.
- The phrase terminates with the new line character `'\n'`.

The function takes the following input:

`words` This is a 2D `char` array that is initially empty. The goal is to fill this array with the words from the read phrase. Each row of `words` will store one such word. The function should work for an arbitrary number of words, but you can assume there are enough rows in `words` to store all the words from the phrase. Also, you need to make sure that each row of `words` is a proper C-string, i.e., it ends with `'\0'`.

It returns the number of words read from the phrase.

(b) (8 points) Write the function `printVertical`. Its purpose is to get an array of words, like the one produced in the above function, and then print them "vertically" next to each other. For instance, when given the array of four words shown below on the left, it should print it in the form shown below on the right, with each word printed like a column. Columns should have one whitespace between them and all words should end at the same height.

```
Coding
is
easier
than
Math
```

```
C   e
o   a
d   s t M
i   i h a
n i e a t
g s r n h
```

The function takes the following two inputs but returns no output (it just prints the words vertically).

`words` This is a 2D `char` array. Each row is a word and it ends with the null termination character `'\0'`. The number of words can be arbitrary, but each word has at most 8 characters plus `'\0'`. The function *should not* modify `words`.

`numWords` The number of words in array `words`.

Running the following `main.cpp` and providing the phrase `"Coding is easier than Math"` should produce the output shown below.

```
int main(){
    char words[30][9];
    int numWords = readAndSplit(words);

    for (int i=0; i < numWords; i++) cout << words[i] << endl;
    printVertical(words,numWords);

    return 0;
}
```

Terminal output:

```
Coding
is
easier
than
Math



C   e
o   a
d   s t M
i   i h a
n i e a t
g s r n h
```

You can use the function `strlen(const char s[])` that returns the length of a C-string `s`, not counting the null termination character `'\0'`. You can also use `cin.getline()` to read a line from the keyboard (see Appendix).

**Problem 7 [20 points] Path Finding in a Maze Using Recursion**

You are to implement recursions for a rat to find a path in a maze represented by an array, given the maze size `MAX_MAZE_SIZE`. Each cell of the input maze array contains one of the two characters:

- `'_'` representing an empty cell where the rat may be placed
- `'#'` representing a wall where the rat is not allowed to be placed

Your functions should try to find a path from a starting cell to an ending cell by moving on the empty cells not exceeding the map boundaries or crossing through walls. If such a path exists, your function should return `true` and mark on the (empty) cells of any one of such paths as `'*'`. Otherwise, the function returns `false` and the `maze` array should not be modified (i.e., kept as is).

In this problem, you **may** define helper functions, but you must **not** use loops in your implementations.

(a) (8 points) Consider a 1-D maze, where the rat can only move right or left horizontally. Implement the following recursive function `solveMaze` which finds a path from a starting cell `startX` to an ending cell `endX`, and draws such a path if exists.

```
/**
 * @brief Solves a 1-D maze by filling in a path from the starting cell
 * `(startX)` to the ending cell`(endX)`.
 *
 * If a path is found, you should replace all cells of the found path
 * with '*' in the `maze` array. Otherwise, the `maze` would not be changed
 * upon the function returns.
 *
 * @param maze   The 1-D array representing the map of the maze.
 * @param startX The X-coordinate of the starting cell.
 * @param endX   The X-coordinate of the ending cell.
 * @return `true` if a path from the starting cell to the ending cell
 *         is found.
 */
bool solveMaze(char maze[MAX_MAZE_SIZE], int startX, int endX);
```

For example, given the following code:

```
#include <iostream>
using namespace std;

const int MAX_MAZE_SIZE = 5;

int main() {
  cout << boolalpha;

  cout << "maze_a:" << endl;
```

```
  char maze_a[MAX_MAZE_SIZE] = {'_', '_', '_', '_', '_'};
  bool pathFound = solveMaze(maze_a, 1, 4);
  cout << pathFound << endl;
  for (int x = 0; x < MAX_MAZE_SIZE; ++x) cout << maze_a[x];
  cout << endl << endl;

  cout << "maze_b:" << endl;
  char maze_b[MAX_MAZE_SIZE] = {'_', '_', '#', '_', '_'};
  pathFound = solveMaze(maze_b, 1, 4);
  cout << pathFound << endl;
  for (int x = 0; x < MAX_MAZE_SIZE; ++x) cout << maze_b[x];
  cout << endl;

  return 0;
}
```

The output of the program will be:

```
maze_a:
true
_****

maze_b:
false
__#__
```

(b) (12 points) Now consider a 2-D maze given by `maze[MAX_MAZE_SIZE][MAX_MAZE_SIZE]`. Implement the following function `solveMaze`, which finds a path from the starting cell at row `startR` and column `startC` to the ending cell at row `endR` and column `endC`. The function returns `true` and draw a path if exists; otherwise it returns `false`.

```
/**
 * @brief Solves the maze by filling in a path from the starting cell
 * at row `startR` and column `startC` to the ending cell at row `endR`
 * and column `endC`.
 *
 * If a path is found, you should replace all cells of the found path
 * with '*' in the `maze` array. Otherwise, the `maze` would not be changed
 * after the function returns.
 *
 * @param maze   The 2-D array representing the map of the maze.
 * @param startR The row index of the starting cell.
 * @param startC The column index of the starting cell.
 * @param endR   The row index of the ending cell.
 * @param endC   The column index of the ending cell.
 * @return `true` if a path from the starting cell to the ending cell
 *         is found.
 */
bool solveMaze(
    char maze[MAX_MAZE_SIZE][MAX_MAZE_SIZE],
    int startR,
    int startC,
    int endR,
    int endC);
```

For example, given the following code:

```
#include <iostream>
using namespace std;

const int MAX_MAZE_SIZE = 4;

int main() {
  cout << boolalpha;

  cout << "maze_a:" << endl;
  char maze_a[MAX_MAZE_SIZE][MAX_MAZE_SIZE] = {
    {'_', '_', '_', '_'},
    {'#', '#', '#', '_'},
    {'#', '_', '_', '_'},
    {'#', '_', '#', '_'},
  };
  bool pathFound = solveMaze(maze_a, 0, 0,3,1);
  cout << pathFound << endl;
  for (int r = 0; r < MAX_MAZE_SIZE; ++r) {
    for (int c = 0; c < MAX_MAZE_SIZE; ++c) cout << maze_a[r][c];
```

```
    cout << endl;
  }

  cout << "maze_b:" << endl;
  char maze_b[MAX_MAZE_SIZE][MAX_MAZE_SIZE] = {
    {'_', '_', '_', '_'},
    {'#', '#', '#', '_'},
    {'#', '_', '_', '#'},
    {'#', '_', '#', '_'},
  };
  pathFound = solveMaze(maze_b, 0, 0,3,1);
  cout << pathFound << endl;
  for (int r = 0; r < MAX_MAZE_SIZE; ++r) {
    for (int c = 0; c < MAX_MAZE_SIZE; ++c) cout << maze_b[r][c];
    cout << endl;
  }
  return 0;
}
```

The output of the program would be:

```
maze_a:
true
****
###*
#***
#*#_

maze_b:
false
____
###_
#__#
#_#_
```

---------- END OF PAPER ----------

## Appendix: Some Library Functions You May Need for Your Answers

```
/* pow() returns the result of the first argument raised to the power of
 * the second argument.
*/

double pow(double base, double exponent);

/* getline reads characters from the input stream and stores them
 * in char_array until it encounters the specified delimiter or until
 * size - 1 characters have been read. The delimiter parameter
 * is optional. The function can also be called with two inputs
 * e.g., cin.getline(arr, 256). In that case the delimiter is
 * assumed to be '\n'.
*/

cin.getline(char[] char_array, unsigned int size, char delimiter);
```